

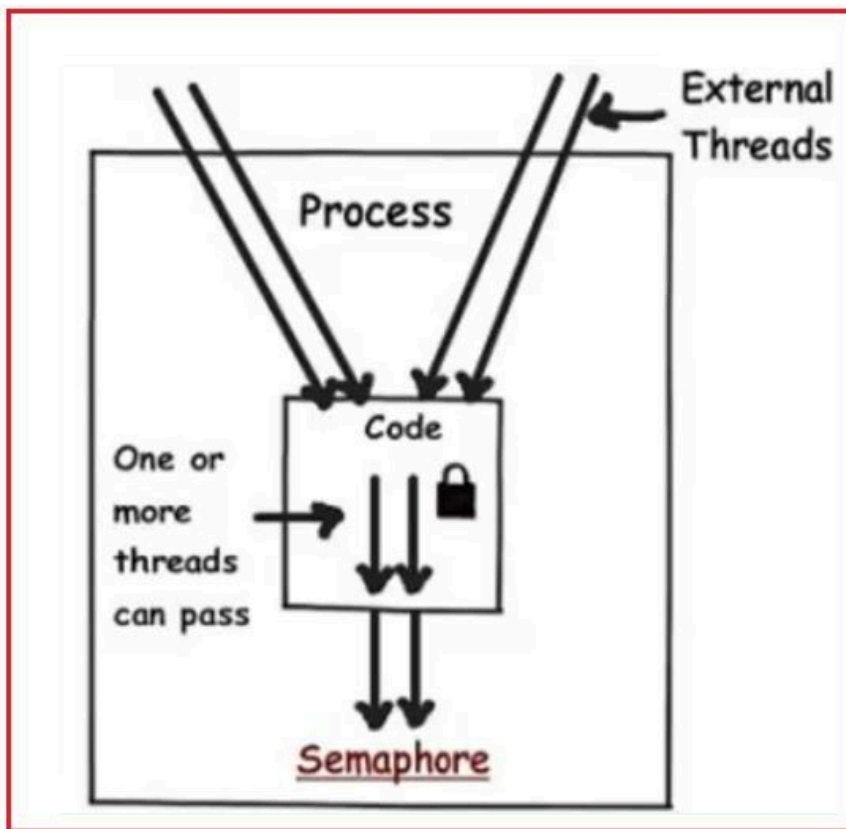
# Semaphore Class in C# with Example

In this article, I am going to discuss **How to Implement Thread Synchronization using Semaphore Class in C#** with Examples. Please read our previous article where we discussed [How to use Mutex in C# to Protect Shared Resources in Multithreading from Concurrent Access](#) with Examples. As part of this article, we are going to discuss the following pointers.

1. **Why do we need Semaphore as we already have Lock, Monitor, and Mutex in C#?**
2. **What is Semaphore in C#?**
3. **How does Semaphore work in C#?**
4. **How to use the Semaphore class?**
5. **Understanding the different methods of Semaphore class in C# with examples.**

## Why do we need Semaphore as we already have Lock, Monitor, and Mutex in C#?

Like Lock, Monitor, and Mutex the Semaphore is also used to provide thread safety in C# Multi-threaded Applications. The Lock and Monitors are basically used to provide thread safety for threads that are generated by the application itself i.e. Internal Threads. On the other hand, Mutex ensures thread safety for threads that are generated by external applications i.e. External Threads. Using Mutex, only one external thread can access our application code at any given point in time and this we have already seen in our previous article. But, if we want more control over the number of external threads that can access our application code, then we need to use the Semaphore class in C#. For a better understanding, please have a look at the below image.



Let us first see an example of how to restrict the number of external threads to access our application code using Semaphore and then we will understand the C# Semaphore Class in detail. In the below example, we

are creating the semaphore instance to allow a maximum of two threads to access our application code i.e. the code in between the WaitOne method and Release method.

```
using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        public static Semaphore semaphore = null;
        static void Main(string[] args)
        {
            try
            {
                //Try to Open the Semaphore if Exists, if not throw an exception
                semaphore = Semaphore.OpenExisting("SemaphoreDemo");
            }
            catch(Exception Ex)
            {
                //If Semaphore not Exists, create a semaphore instance
                //Here Maximum 2 external threads can access the code at the same time
                semaphore = new Semaphore(2, 2, "SemaphoreDemo");
            }
            Console.WriteLine("External Thread Trying to Acquiring");
            semaphore.WaitOne();
            //This section can be access by maximum two external threads: Start
            Console.WriteLine("External Thread Acquired");
            Console.ReadKey();
            //This section can be access by maximum two external threads: End
            semaphore.Release();
        }
    }
}
```

Now, build the project and then run the application EXE file three times. The first two times, you will see the message that External Thread Acquired, but when you run the EXE file for the 3rd time, you will see the message External Thread Trying to Acquiring only as shown in the below image.

## First Run

 C:\Users\HP\source\repos\MutexDemo\SemaphoreDemo\bin\Debug\SemaphoreDemo.exe

```
External Thread Trying to Acquiring  
External Thread Acquired
```

## Second Run

 C:\Users\HP\source\repos\MutexDemo\SemaphoreDemo\bin\Debug\SemaphoreDemo.exe

```
External Thread Trying to Acquiring  
External Thread Acquired
```

## Third Run

 C:\Users\HP\source\repos\MutexDemo\SemaphoreDemo\bin\Debug\SemaphoreDemo.exe

```
External Thread Trying to Acquiring
```

Now, I hope you understand the basic need for the Semaphore Class in C#. Let us proceed further and understand the C# Semaphore Class in detail.

## What is Semaphore Class in C#?

The Semaphore Class in C# is used to limit the number of external threads that can have access to a shared resource concurrently. In other words, we can say that Semaphore allows one or more external threads to enter into the critical section and execute the task concurrently with thread safety. So, in real-time, we need to use Semaphore when we have a limited number of resources and we want to limit the number of threads that can use it.

## Constructors, and Methods of Semaphore Class in C#:

Let us understand the different Constructors, and Methods of Semaphore Class in C#. If you right-click on the Semaphore class and select go to definition, then you will see the following. Here, you can see that Semaphore is a sealed class and it is inherited from WaitHandle class. As a sealed class, no further inheritance is possible.

```
...public sealed class Semaphore : WaitHandle  
{  
    ...public Semaphore(int initialCount, int maximumCount);  
    ...public Semaphore(int initialCount, int maximumCount, string name);  
    ...public Semaphore(int initialCount, int maximumCount, string name, out bool createdNew);  
    ...public Semaphore(int initialCount, int maximumCount, string name, out bool createdNew, SemaphoreSecurity semaphoreSecurity);  
  
    ...public static Semaphore OpenExisting(string name);  
    ...public static Semaphore OpenExisting(string name, SemaphoreRights rights);  
    ...public static bool TryOpenExisting(string name, out Semaphore result);  
    ...public static bool TryOpenExisting(string name, SemaphoreRights rights, out Semaphore result);  
    ...public SemaphoreSecurity GetAccessControl();  
    ...public int Release();  
    ...public int Release(int releaseCount);  
    ...public void SetAccessControl(SemaphoreSecurity semaphoreSecurity);  
}
```

## Constructors of Semaphore Class in C#:

The Semaphore Class in C# provides the following four constructors that we can use to create an instance of the Semaphore class.

1. **Semaphore(int initialCount, int maximumCount)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries.
2. **Semaphore(int initialCount, int maximumCount, string name)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries, and optionally specifying the name of a system semaphore object.
3. **Semaphore(int initialCount, int maximumCount, string name, out bool createdNew)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries, optionally specifying the name of a system semaphore object, and specifying a variable that receives a value indicating whether a new system semaphore was created.
4. **Semaphore(int initialCount, int maximumCount, string name, out bool createdNew, SemaphoreSecurity semaphoreSecurity)**: It initializes a new instance of the Semaphore class, specifying the initial number of entries and the maximum number of concurrent entries, optionally specifying the name of a system semaphore object, specifying a variable that receives a value indicating whether a new system semaphore was created, and specifying security access control for the system semaphore.

Parameters used in Semaphore Class Constructors:

1. **initialCount**: The initial number of requests for the semaphore that can be granted concurrently. It throws `ArgumentException` if `initialCount` is greater than `maximumCount`.
2. **maximumCount**: The maximum number of requests for the semaphore that can be granted concurrently. It will throw `ArgumentOutOfRangeException` if `maximumCount` is less than 1 or `initialCount` is less than 0.
3. **name**: The name of a named system semaphore object.
4. **createdNew**: When this method returns, contains true if a local semaphore was created (that is, if the name is null or an empty string) or if the specified named system semaphore was created; false if the specified named system semaphore already existed. This parameter is passed uninitialized.
5. **semaphoreSecurity**: A `System.Security.AccessControl.SemaphoreSecurity` object that represents the access control security to be applied to the named system semaphore.

## Methods of Semaphore Class in C#:

The Semaphore Class in C# provides the following methods.

1. **OpenExisting(string name)**: This method is used to open a specified named semaphore if it already exists. It returns an object that represents the named system semaphore. Here, the parameter name specifies the name of the system semaphore to open. It will throw `ArgumentException` if the name is an empty string. -or- name is longer than 260 characters. It will throw `ArgumentNullException` if the name is null.
2. **OpenExisting(string name, SemaphoreRights rights)**: This method is used to open the specified named semaphore, if it already exists, with the desired security access. It returns an object that represents the named system semaphore. Here, the parameter name specifies the name of the system semaphore to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access.
3. **TryOpenExisting(string name, out Semaphore result)**: This method is used to open the specified named Semaphore, if it already exists, and returns a value that indicates whether the operation

succeeded. Here, the parameter name specifies the name of the system Semaphore to open. When this method returns, the result contains a Semaphore object that represents the named Semaphore if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named mutex was opened successfully; otherwise, false.

4. **TryOpenExisting(string name, SemaphoreRights rights, out Semaphore result)**: This method is used to open the specified named Semaphore, if it already exists, with the desired security access, and returns a value that indicates whether the operation succeeded. Here, the parameter name specifies the name of the system Semaphore to open. The parameter rights specify a bitwise combination of the enumeration values that represent the desired security access. When this method returns, the result contains a Semaphore object that represents the named Semaphore if the call succeeded, or null if the call failed. This parameter is treated as uninitialized. It returns true if the named Semaphore was opened successfully; otherwise, false.
5. **Release()**: This method exit the semaphore and returns the previous count. It returns the count on the semaphore before the Release method was called.
6. **Release(int releaseCount)**: This method exit the semaphore a specified number of times and returns the previous count. Here, the parameter releaseCount specifies the number of times to exit the semaphore. It returns the count on the semaphore before the Release method was called.
7. **GetAccessControl()**: This method Gets the access control security for a named system semaphore.
8. **SetAccessControl(SemaphoreSecurity semaphoreSecurity)**: This method Sets the access control security for a named system semaphore.

**Note:** The **Semaphore** Class in C# is inherited from **WaitHandle** abstract class and the WaitHandle class provides the WaitOne() method which we need to call to lock the resource. Note that a Semaphore object can only be released from the same thread which obtained it.

1. **WaitOne() Method**: Threads can enter into the critical section by using the WaitOne method. We need to call the WaitOne method on the semaphore object. If the Int32 variable which is maintained by semaphore is greater than 0 then it allows the thread to enter into the critical section.

## How does Semaphore work in C#?

The Semaphores are Int32 variables that are stored in operating system resources. When we initialize the semaphore object we initialize it with a number. This number basically used to limit the threads that can enter the critical section.

So, when a thread enters the critical section, it decreases the value of the Int32 variable by 1 and when a thread exits from the critical section, it then increases the value of the Int32 variable by 1. The most important point that you need to remember is when the value of the Int32 variable is 0, then no thread can enter into the critical section.

## How to Create a Semaphore in C#?

You can use the following statement to create the Semaphore instance in C#. Here, we use the overloaded version of the constructor which takes two parameters to create an instance of the semaphore class.

**Semaphore semaphoreObject = new Semaphore(initialCount: 2, maximumCount: 3);**

As you can see in the above statement, we are passing two values to the Constructor of the Semaphore class while initializing. These two values represent InitialCount and MaximumCount. The maximumCount defines how many maximum threads can enter the critical section and the initialCount set the value of the Int32 variable.

The InitialCount parameter sets the value for the Int32 variable. That is it defines the initial number of requests for the semaphore that can be granted concurrently. The MaximumCount parameter defines the maximum number of requests for the semaphore that can be granted concurrently.

For example, if we set the maximum count value as 3 and the initial count value is 0, it means 3 threads are already in the critical section, so no more new threads can enter the critical section. If we set the maximum count value as 3 and the initial count value as 2. It means a maximum of 3 threads can enter the critical section and there is one thread that is currently in the critical section, so two new threads can enter the critical section.

**Note1:** When a thread enters the critical section, it decreases the initialCount variable value by 1 and when a thread exits from the critical section, it then increases the initialCount variable value by 1. And when the value of the initialCount variable is 0, then no thread can enter into the critical section. The second parameter maximumCount always must be equal to or greater than the first parameter initialCount otherwise we will get an exception.

**Note2:** We need to call the Release() method when the thread wants to exit from the critical section. When this method is called, it increments the Int32 variable which is maintained by the semaphore object.

## Example to understand Semaphore in C#:

Let us see an example for a better understanding of how to use the Semaphore to implement Thread Synchronization to Protect shared resources in multithreading from concurrent access in C#. Please have a look at the below example. In the below example, we initialize a semaphore object with 2 initialcount and maximum of 3 threads that can enter the critical section. We start the for loop with runs from 0 to 10. We started threads using the Thread class and the call shared resource DoSomeTask method.

Each thread calls the WaitOne method of semaphore object before doing the required task. The WaitOne method will decrease the initialcount variable value by 1. So, the WaitOne method will limit the number of threads to access the shared resource. After completing the task each thread calls the Release method which will increment initialcount variable value by 1 of the semaphore object. This allows further threads to enter into a critical section.

```
using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        public static Semaphore semaphore = new Semaphore(2, 3);
        static void Main(string[] args)
        {
            for (int i = 1; i <= 10; i++)
            {
                Thread threadObject = new Thread(DoSomeTask)
                {
                    Name = "Thread " + i
                };
                threadObject.Start();
            }
            Console.ReadKey();
        }
    }
}
```

```

    }
    static void DoSomeTask()
    {
        Console.WriteLine(Thread.CurrentThread.Name + " Wants to Enter into
Section for processing");
        try
        {
            //Blocks the current thread until the current WaitHandle receives
signal.
            semaphore.WaitOne();
            //Decrease the Initial Count Variable by 1
            Console.WriteLine("Success: " + Thread.CurrentThread.Name + "
its work");
            Thread.Sleep(5000);
            Console.WriteLine(Thread.CurrentThread.Name + "Exit.");
        }
        finally
        {
            //Release() method to release semaphore
            //Increase the Initial Count Variable by 1
            semaphore.Release();
        }
    }
}
}

```

**Output:**



```
Success: Thread 3 is Doing its work
Thread 1 Wants to Enter into Critical Section for processing
Thread 5 Wants to Enter into Critical Section for processing
Thread 2 Wants to Enter into Critical Section for processing
Success: Thread 1 is Doing its work
Thread 4 Wants to Enter into Critical Section for processing
Thread 6 Wants to Enter into Critical Section for processing
Thread 7 Wants to Enter into Critical Section for processing
Thread 9 Wants to Enter into Critical Section for processing
Thread 8 Wants to Enter into Critical Section for processing
Thread 10 Wants to Enter into Critical Section for processing
Thread 3Exit.
Thread 1Exit.
Success: Thread 4 is Doing its work
Success: Thread 2 is Doing its work
Thread 4Exit.
Success: Thread 5 is Doing its work
Thread 2Exit.
Success: Thread 6 is Doing its work
Thread 5Exit.
Success: Thread 7 is Doing its work
Thread 6Exit.
Success: Thread 8 is Doing its work
Thread 7Exit.
Success: Thread 9 is Doing its work
Thread 8Exit.
Success: Thread 10 is Doing its work
Thread 9Exit.
Thread 10Exit.
```

As you can see in the above output, here maximum of two threads are entering the critical section and doing their tasks at any given point in time.

In the next article, I am going to discuss [How to Implement Thread Synchronization using SemaphoreSlim Class in C#](#) with Examples. Here, in this article, I try to explain **How to Implement Thread Synchronization using Semaphore Class in C#** with Examples. I hope you enjoy this article and understand the concept of Semaphore Class in C# with Examples.